# Constella Data Layer

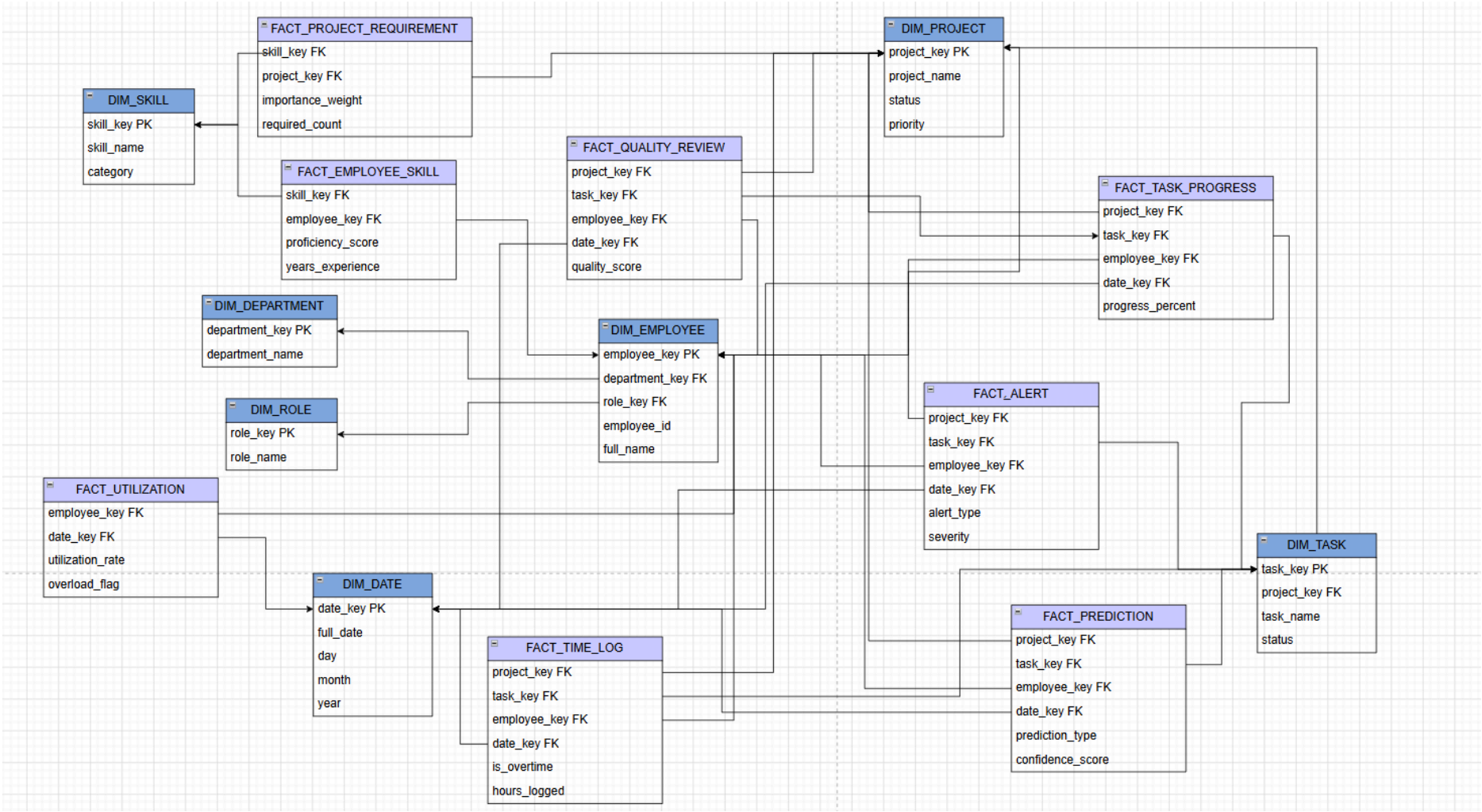
## DW Architecture

| Layer | Purpose |
|---|---|
| Public | operational data |
| Staging | raw copy of data |
| Dw | warehouse (facts + dimensions) |
| Mart (views) | dashboards |

**Why Staging?**

- We do not touch production tables directly.
- We can clean/transform data before loading DW.
- Refresh becomes controlled.
- If something goes wrong, public data is still safe.

# Schema Design




- Constellation + Snowflake Schema

## Fact tables

1. **Time_log**
    - **Stores:**
        - employee
        - project
        - task
        - date
        - hours_logged
        - Is_overtime
    - **Dashboard use:**
        - Total hours worked
        - Hours by employee
        - Hours by project
        - Hours by task
        - Overtime analysis
        - Productivity tracking
    - **Example KPI:**
        - Total logged hours per project
    - **Useful charts:**
        - Bar chart: hours by project
        - Line chart: hours over time
        - Table: employee time contribution

2. **Utilization:**
    - **Stores: workload/utilization metrics**
        - employee
        - date
        - available_hours
        - logged_hours
        - utilization_rate
        - overload_flag
        - underutilized_flag
    - **Dashboard use:**
        - Employee workload
        - Overloaded employees
        - Underutilized employees
        - Utilization rate trends
        - Team capacity planning
    - **Example KPI:**
        - Average Utilization Rate : SELECT AVG(utilization_rate) FROM dw.fact_utilization;
        - Overload Percentage : SELECT COUNT(*) FILTER (WHERE overload_flag = TRUE)::float / COUNT(*) * 100 AS overload_rate FROM dw.fact_utilization;
        - Underutilization Percentage SELECT COUNT(*) FILTER (WHERE underutilized_flag = TRUE)::float / COUNT(*) * 100 AS underutilized_rate FROM dw.fact_utilization;
    - **Useful charts:**
        - Gauge: average utilization
        - Table: overloaded employees

        - Line chart: utilization over time

3. **Employee_skill**
    - **Stores:**
        - employee
        - skill
        - department
        - role
        - proficiency_score
        - years_experience
        - skill_level
    - **Dashboard use:**
        - Skill distribution
        - Skill gap analysis
        - Best employees for a skill
        - Department skill strength
        - Role-based skill comparison
    - **Example KPI:**
        - Average Skill Proficiency
        - How many skills are covered by employees

```
SELECT
  COUNT(DISTINCT skill_key)::float /
  (SELECT COUNT(*) FROM dw.dim_skill) * 100 AS coverage
FROM dw.fact_employee_skill;
```

        - Skill Gap

```
SELECT pr.skill_key
```

```
FROM dw.fact_project_requirement pr
LEFT JOIN dw.fact_employee_skill es
ON pr.skill_key = es.skill_key
WHERE es.skill_key IS NULL;
```

- **Useful charts:**
  - Heatmap: skills by department
  - Leaderboard: top employees by skill
  - Bar chart: skill levels count

**4. Project_requirement**

- **Stores:**
  - project
  - skill
  - required_level
  - required_count
  - importance_weight
- **Dashboard use:**
  - Project skill needs
  - Skill shortages
  - Staffing recommendations
  - Project readiness
- **Example KPI:**
  - Skill Demand per Project”complexity of the project”

```
SELECT project_key, COUNT(*) AS required_skills
FROM dw.fact_project_requirement
GROUP BY project_key;
```

        - Skill Importance Score

          SELECT AVG(importance_weight) FROM dw.fact_project_requirement;
        - Resource Sufficiency “Do we have enough people? ”

          SELECT
            pr.project_key,
            COUNT(es.employee_key) AS available,
            SUM(pr.required_count) AS required
          FROM dw.fact_project_requirement pr
          LEFT JOIN dw.fact_employee_skill es
          ON pr.skill_key = es.skill_key
          GROUP BY pr.project_key;
    - **Useful charts:**
        - Required skills per project
        - Missing skills by project
        - Skill demand across projects
5. **Task_progress:**
    - **Stores: task status changes over time**
        - task
        - project
        - changed_by_employee
        - progress_percent
        - old_status
        - new_status
        - date
    - **Dashboard use:**

        - Task progress over time
        - Status movement
        - Bottlenecks
        - Completion trends
        - Delayed tasks
    - **Example KPI:**
        - Average Task Progress “overall project progress ”

          SELECT AVG(progress_percent) FROM dw.fact_task_progress;
        - Task Completion Rate

          SELECT
            COUNT(CASE WHEN new_status = 'completed' THEN 1 END)::float
            / COUNT(*) * 100 AS completion_rate
          FROM dw.fact_task_progress;
        - Stuck Tasks (Bottlenecks)

          SELECT task_key, COUNT(*) AS updates
          FROM dw.fact_task_progress
          GROUP BY task_key
          HAVING COUNT(*) > 10;
    - **Useful charts:**
        - Line chart: average task progress over time
        - Stacked chart: task status counts
        - Table: tasks stuck in same status
6. Quality_review
    - **Stores: quality and productivity review data.**
        - project
        - task

        - reviewer
        - reviewed_employee
        - quality_score
        - productivity_score
        - defects_found
        - rework_required
    - **Dashboard use:**
        - Employee quality score
        - Productivity score
        - Defect tracking
        - Rework frequency
        - Quality trends
    - **Example KPI:**
        - Average Quality Score”Overall quality ”

          SELECT AVG(quality_score) FROM dw.fact_quality_review;
        - Defect Rate “Quality issues ”

          SELECT AVG(defects_found) FROM dw.fact_quality_review;
        - Rework Percentage “Inefficiency indicator ”

          SELECT
            COUNT(*) FILTER (WHERE rework_required = TRUE)::float
            / COUNT(*) * 100 AS rework_rate
          FROM dw.fact_quality_review;
    - **Useful charts:**
        - Average quality score by employee
        - Defects by project

        - Rework required percentage
7. Alert:
    - **Stores:**
        - project
        - task
        - employee
        - alert_type
        - severity
        - status
        - alert_count
    - **Dashboard use:**
        - Open alerts
        - Resolved alerts
        - High severity risks
        - Employee-level issues
        - Project-level warnings
    - **Example KPI:**
        - Total Alerts”System health”

          SELECT COUNT(*) FROM dw.fact_alert;
        - High Severity Alerts”Critical risks ”

          SELECT COUNT(*)
          FROM dw.fact_alert
          WHERE severity = 'high';

        - Open Alerts Ratio”Unresolved issues”

```
SELECT
  COUNT(*) FILTER (WHERE status = 'open')::float
  / COUNT(*) * 100 AS open_alerts
FROM dw.fact_alert;
```

- **Useful charts:**
  - Alert count by severity
  - Open alerts by project
  - Alert type distribution

8. Prediction
   - **Stores: AI/model outputs**
     - employee/project/task
     - prediction_type
     - predicted_value
     - confidence_score
     - target_date
     - model_version
   - **Dashboard use:**
     - Delay risk
     - Burnout risk
     - Budget overrun risk
     - Prediction confidence
     - Model output tracking
   - **Example KPI:**
     - Average Risk Score “Overall risk level ”

```
SELECT AVG(predicted_value) FROM dw.fact_prediction;
```

        - High Risk Predictions”Risk alerts (as burnout/delay) ”

          ```
          SELECT COUNT(*)
          FROM dw.fact_prediction
          WHERE predicted_value > 0.7;
          ```

        - Model Confidence

          ```
          SELECT AVG(confidence_score) FROM dw.fact_prediction;
          ```

    - **Useful charts:**
        - Risk distribution (histogram)
        - High-risk projects table
        - Confidence trend over time
9. Monitoring_indicator
    - **Stores: KPI threshold monitoring**
        - project
        - employee
        - indicator_name
        - indicator_value
        - threshold_value
        - status
        - date
    - **Dashboard use:**
        - KPI health
        - Threshold warnings
        - Project monitoring
        - Employee monitoring
    - **Example KPI:**

- KPI Breach Rate”How often KPIs fail ”

```
SELECT
  COUNT(*) FILTER (WHERE indicator_value > threshold_value)::float
  / COUNT(*) * 100 AS breach_rate
FROM dw.fact_monitoring_indicator;
```

- Healthy Indicators”Stable performance ”

```
SELECT COUNT(*)
FROM dw.fact_monitoring_indicator
WHERE status = 'healthy';
```

- Warning Indicators “Early risk detection ”

```
SELECT COUNT(*)
FROM dw.fact_monitoring_indicator
WHERE status = 'warning';
```

- **Useful charts:**
  - KPI status cards
  - Threshold exceeded alerts
  - Indicator trend by project

10. Trend_indicator
    - **Stores:**
      - entity_type
      - entity_id
      - metric_name
      - metric_value
      - trend_direction
      - trend_percent
      - date

- **Dashboard use:**
  - Performance improving/declining/stable
  - Trend by employee
  - Trend by project
  - Trend by department
- **Example KPI:**
  - Improvement Rate

```
SELECT
  COUNT(*) FILTER (WHERE trend_direction = 'up')::float
  / COUNT(*) * 100 AS improvement_rate
FROM dw.fact_trend_indicator;
```

  - Decline Rate “Performance issues ”

```
SELECT
  COUNT(*) FILTER (WHERE trend_direction = 'down')::float
  / COUNT(*) * 100 AS decline_rate
FROM dw.fact_trend_indicator;
```

  - Average Trend % “Strength of trends ”

```
SELECT AVG(trend_percent) FROM dw.fact_trend_indicator;
```

- **Useful charts:**
  - Trend arrows
  - Metric movement over time
  - Top declining projects/employees

**Dimension Tables**

- dim_date
- dim_employee
- dim_department

- dim_role
- dim_skill
- dim_project
- dim_task

So how does the refreshment get handled?

1. dw.refresh_staging() → Copy latest data from public into staging.
    - **TRUNCATE staging.stg_x;** → delete old staging copy
    - **INSERT INTO staging.stg_x**
    - **SELECT * FROM public.x;** → copy latest public data
2. dw.refresh_dimensions() →Load/update dimension tables.
    - **ON CONFLICT DO UPDATE** → dimensions stay updated without duplication.
3. dw.refresh_facts() → Rebuild all analytical facts
    - TRUNCATE all facts
    - INSERT fresh fact rows from staging + dimensions
4. dw.refresh_dw() → wrapper/orchestrator

```
CREATE OR REPLACE FUNCTION dw.refresh_dw()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    PERFORM dw.refresh_staging();
    PERFORM dw.refresh_dimensions();
    PERFORM dw.refresh_facts();
END;
$$;
```

- We only calls **SELECT dw.refresh_dw();** And the full pipeline runs

## May help in the backend:

### Dw_refresh.py

```
from database import get_connection
def refresh_dw():
    conn = get_connection()
    try:
        with conn:
            with conn.cursor() as cur:
                cur.execute("SELECT dw.refresh_dw();")
        return {"status": "success", "message": "Data warehouse refreshed"}
    except Exception as e:
        return {"status": "error", "message": str(e)}
    finally:
        conn.close()
```

### main.py

```
from fastapi import FastAPI
from dw_refresh import refresh_dw
app = FastAPI()
@app.post("/dw/refresh")
def refresh_data_warehouse():
    return refresh_dw()
```

```
@app.get("/dashboard")
def view_dashboard():
    refresh_result = refresh_dw()
    return {
        "message": "Dashboard opened",
        "refresh": refresh_result
    }
```

## Refresh when project ends

```
from fastapi import FastAPI
from database import get_connection
from dw_refresh import refresh_dw

app = FastAPI()

@app.put("/projects/{project_id}/status")
def update_project_status(project_id: int, status: str):
    conn = get_connection()

    try:
        with conn:
            with conn.cursor() as cur:
                cur.execute(
                    """
                    UPDATE public.project
                    SET status = %s,
                        updated_at = NOW()
                    WHERE project_id = %s
                    """,
                    (status, project_id)
                )
```

```
    if status.lower() in ["completed", "ended", "finished"]:
        refresh_dw()

    return {
        "message": "Project status updated",
        "dw_refreshed": status.lower() in ["completed", "ended", "finished"]
    }

finally:
    conn.close()
```